



**ÉCOLE MAROCAINE DES
SCIENCES DE L'INGÉNIEUR**
Membre de
HONORIS UNITED UNIVERSITIES

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

EMSI Casablanca

Filière : Ingénierie Informatique et Réseaux – 3IIR

Année universitaire : 2025/2026

RAPPORT DE PROJET DE FIN D'ANNÉE

FakeFinder

Système Intelligent de Détection du Hameçonnage par E-mail

Réalisé par :

Yahya KHALIFI

Elias CHAKROUN

Tuteur académique :

Mme Soukayna MAYARA

Date de soutenance :

18 Mai 2026

Remerciements

Nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nos remerciements s'adressent en premier lieu à notre encadrante, **Madame Soukayna MAYARA**, pour ses conseils avisés, ses orientations pertinentes, sa disponibilité et son suivi rigoureux tout au long de ce projet de fin d'année. Ses directives nous ont permis de structurer notre travail de manière méthodique et scientifique.

Nous remercions également le corps professoral et l'administration de **l'École Marocaine des Sciences de l'Ingénieur (EMSI)** pour la qualité de l'enseignement dispensé tout au long de notre cursus. Les connaissances acquises au sein de l'établissement ont été le socle indispensable à la réalisation technique de ce projet.

Enfin, nous adressons nos remerciements les plus chaleureux à nos familles et à nos amis pour leur soutien moral indéfectible, leur patience et leurs encouragements durant nos études.

Résumé

FakeFinder est une solution de sécurité complète basée sur le framework Django, conçue pour lutter contre la menace croissante du hameçonnage (phishing) par e-mail. Face à la sophistication des attaques d'ingénierie sociale, les filtres antispam traditionnels s'avèrent souvent insuffisants. En s'appuyant sur des modèles d'apprentissage automatique, spécifiquement l'algorithme Random Forest couplé à une vectorisation TF-IDF (Term Frequency-Inverse Document Frequency), ce système effectue une analyse sémantique et structurelle approfondie des en-têtes et du corps des e-mails. L'application offre aux utilisateurs une interface web intuitive permettant de vérifier la légitimité des fichiers .eml de manière quasi instantanée, tout en appliquant une politique stricte de "Zero Retention" pour garantir la confidentialité absolue des données traitées. Ce rapport détaille le processus de conception architecturale, la phase d'ingénierie des caractéristiques (Feature Engineering), l'implémentation technique, ainsi que l'évaluation des performances du modèle. Il explore également les évolutions futures de la plateforme, notamment l'intégration prévue de l'API VirusTotal pour une vérification dynamique et exhaustive de la réputation des URL.

Mots-clés : hameçonnage, détection, apprentissage automatique, Random Forest, TF-IDF, VirusTotal, Django, cybersécurité, Zero Retention.

Abstract

FakeFinder is a comprehensive Django-based security solution designed to combat the growing threat of email phishing. As social engineering attacks become increasingly sophisticated, traditional spam filters often prove insufficient. By leveraging machine learning models—specifically the Random Forest algorithm coupled with TF-IDF (Term Frequency-Inverse Document Frequency) vectorization—this system performs an in-depth semantic and structural analysis of both email headers and body content. The application provides users with an intuitive web interface to verify the legitimacy of .eml files almost instantaneously, while enforcing a strict "Zero Retention" policy to ensure absolute data privacy. This report details the architectural design process, the feature engineering phase, technical implementation, and the performance evaluation of the model. It also explores future developments for the platform, notably the planned integration of the VirusTotal API for dynamic and exhaustive URL reputation verification.

Keywords : phishing, detection, machine learning, Random Forest, TF-IDF, VirusTotal, Django, cybersecurity, Zero Retention.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Table des figures	vi
Liste des tableaux	vii
1 Introduction Générale	1
1.1 Contexte et Motivation	1
1.2 Énoncé du Problème	1
1.3 Objectifs du Projet	2
2 Présentation de l'Organisme et du Contexte	3
2.1 Présentation de l'EMSI	3
2.2 Le Phishing : Évolution et Taxonomie de la Menace	4
2.2.1 Le Phishing de Masse (Deceptive Phishing)	4
2.2.2 Le Spear-Phishing (Harponnage)	4
2.2.3 Le Whaling (Chasse à la baleine)	5
2.2.4 Les Techniques d'Évasion Modernes	5
3 Spécification et Gestion du Projet	6
3.1 Exigences Fonctionnelles du Système	6
3.2 Exigences Non-Fonctionnelles (NFR)	7
3.2.1 Contraintes de Sécurité et Confidentialité	7
3.2.2 Performances et Temps de Réponse	7
3.3 Planification et Répartition des Tâches	7
4 Analyse et Conception du Système	9
4.1 Modélisation des Processus Métier	9
4.1.1 Diagramme de Cas d'Utilisation	9
4.2 Modélisation des Données et Architecture	10

4.2.1	Architecture Globale (MVT)	10
4.2.2	Diagramme de Classes	10
5	Réalisation, Modélisation ML et Développement	12
5.1	Environnement Technologique et Outils	12
5.2	Le Modèle de Machine Learning : Traitement et Entraînement	13
5.2.1	Préparation des Données et Extraction de Caractéristiques	13
5.2.2	Choix et Entraînement de l'Algorithme : Random Forest	14
5.3	Interfaces et Expérience Utilisateur (Frontend)	15
6	Perspectives Technologiques : Intégration de VirusTotal	17
6.1	Limites de l'Approche Heuristique Pure	17
6.2	Architecture de l'Intégration VirusTotal (Évolution Prévue)	17
7	Tests, Évaluation et Conclusion	19
7.1	Méthodologie de Test et Validation du Modèle	19
7.2	Tests de Charge et Fonctionnels de la Plateforme Web	19
7.3	Conclusion Générale	20
	Bibliographie	21
A	Listings du Code Source	22
A.1	Caractéristiques d'Apprentissage Automatique (ml/features.py)	22
A.2	Entraînement de l'Apprentissage Automatique (ml/train.py)	27
A.3	Modèles du Scanner (scanner/models.py)	32
A.4	Vues du Scanner (scanner/views.py)	33
A.5	Utilitaires du Scanner (scanner/utls.py)	40
A.6	Middleware du Scanner (scanner/middleware.py)	43

Table des figures

2.1	Structure organisationnelle et infrastructures de l'EMSI	4
4.1	Diagramme de cas d'utilisation global du système FakeFinder	10
4.2	Diagramme de Classes détaillant les entités du domaine	11
5.1	Environnement de développement (VS Code) illustrant la structure backend . .	13
5.2	Extrait de l'implémentation du pipeline de transformation ML en Python	14
5.3	Page d'accueil : Interface d'upload "Drag and Drop"	15
5.4	Tableau de bord des résultats d'analyse	16

Liste des tableaux

3.1	Tableau des Exigences Fonctionnelles (FR)	6
3.2	Planning de réalisation du PFA (Méthodologie Agile)	8

Chapitre 1

Introduction Générale

1.1 Contexte et Motivation

Dans le paysage numérique moderne, la messagerie électronique demeure la pierre angulaire de la communication professionnelle et personnelle. Avec des milliards d'e-mails échangés quotidiennement à travers le monde, ce canal est naturellement devenu la cible privilégiée des cybercriminels. Le hameçonnage (ou phishing), une forme pernicieuse d'ingénierie sociale, consiste à usurper l'identité d'une entité de confiance (banque, administration, collègue) dans le but de soutirer des informations sensibles (identifiants, données bancaires) ou de déployer des logiciels malveillants.

Au cours des dernières années, le hameçonnage a évolué de simples arnaques de masse vers des campagnes hautement ciblées (spear-phishing) et sophistiquées. Les attaquants utilisent désormais des techniques d'obfuscation avancées, contournant allègrement les filtres de sécurité traditionnels des entreprises.

La motivation derrière le projet **FakeFinder** découle directement de ce constat alarmant. Il existe un besoin critique pour un outil accessible, indépendant et puissant, permettant à tout utilisateur de vérifier un e-mail suspect avant d'interagir avec son contenu. FakeFinder se positionne comme un système de "deuxième avis" diagnostique, effectuant une analyse approfondie et hors-ligne des fichiers de messagerie bruts (au format .eml).

1.2 Énoncé du Problème

La sécurité traditionnelle des e-mails s'appuie historiquement sur des approches basées sur des listes noires (URL malveillantes connues) ou sur la vérification de la réputation des adresses IP des expéditeurs. Ces méthodes, bien que nécessaires, sont intrinsèquement réactives. Elles s'avèrent inefficaces contre les attaques de type "zero-day" (nouvelles menaces non répertoriées) ou lorsque des attaquants compromettent des comptes légitimes pour envoyer leurs messages frauduleux.

De plus, la majorité des utilisateurs finaux ne disposent ni des compétences techniques ni du temps nécessaire pour inspecter manuellement les en-têtes de routage d'un e-mail (champs Received, Return-Path) à la recherche de signaux d'usurpation. Il est donc impératif de

concevoir un outil automatisant ce processus d'investigation complexe, pour le restituer sous la forme d'un indicateur de risque clair, binaire et exploitable.

1.3 Objectifs du Projet

L'objectif fondamental de ce Projet de Fin d'Année est de développer un système hybride de détection de hameçonnage, intégrant l'Intelligence Artificielle à l'ingénierie logicielle classique. Les objectifs spécifiques se déclinent comme suit :

- **Traitement des données brutes** : Développer un moteur d'analyse syntaxique (parser) performant, capable d'extraire en mémoire vive le corps du texte, les liens hyperboliques et les métadonnées d'un fichier .eml, indépendamment de son encodage.
- **Intégration de l'Apprentissage Automatique** : Concevoir, entraîner et déployer un classificateur basé sur l'algorithme Random Forest, optimisé pour la détection sémantique de textes frauduleux via la méthode TF-IDF.
- **Évolutivité de l'Analyse des Liens** : Préparer l'architecture logicielle à l'intégration future de l'API VirusTotal, permettant de croiser les prédictions de l'IA avec des bases de données mondiales de cybermenaces.
- **Développement Web Full-Stack** : Créer une plateforme web ergonomique sous Django, simplifiant au maximum l'expérience utilisateur (glisser-déposer de fichiers).
- **Garantie de Confidentialité (Zero Retention)** : Assurer que le système traite les données de manière éphémère. Aucun fichier soumis ne doit être persisté sur les disques du serveur, garantissant ainsi le respect de la vie privée et de la réglementation sur les données.

Chapitre 2

Présentation de l'Organisme et du Contexte

2.1 Présentation de l'EMSI

L'École Marocaine des Sciences de l'Ingénieur (EMSI) est un établissement privé d'enseignement supérieur fondé en 1986. Reconnue par l'État, l'EMSI s'est imposée au fil des décennies comme la plus grande école d'ingénieurs privée au Maroc, formant des cadres pluridisciplinaires capables d'accompagner le développement technologique et industriel du Royaume.

Membre fondateur du réseau panafricain *Honoris United Universities*, l'école met un point d'honneur à offrir une pédagogie axée sur l'innovation, la pratique et l'ouverture à l'international. L'EMSI structure ses formations autour de plusieurs filières d'ingénierie d'excellence, dont la filière Ingénierie Informatique et Réseaux (IIR). Cette dernière vise à doter les futurs ingénieurs de compétences pointues en développement de systèmes d'information, en administration réseaux, en intelligence artificielle et en cybersécurité.

C'est dans le cadre de la troisième année de la filière IIR (3IIR) qu'intervient ce Projet de Fin d'Année (PFA), constituant une mise en pratique concrète des modules théoriques étudiés au cours de l'année.

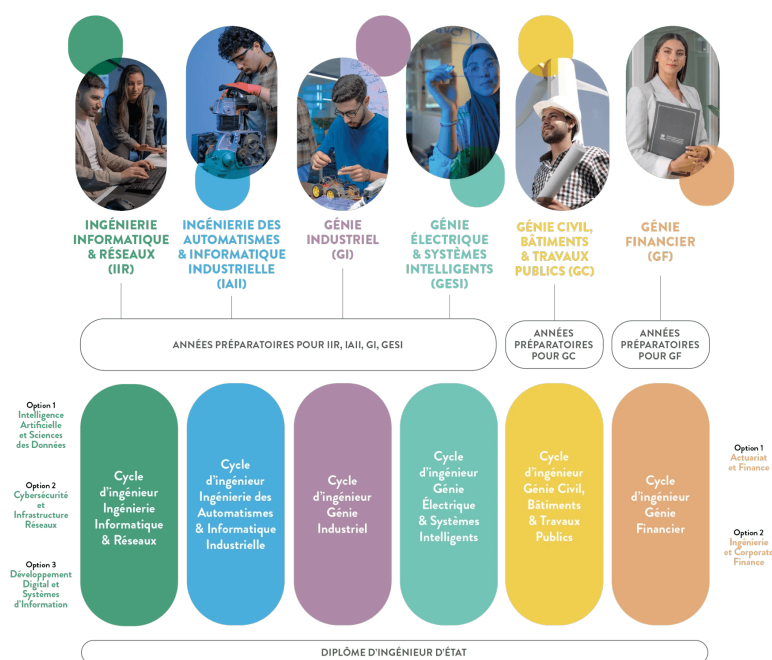


FIGURE 2.1 – Structure organisationnelle et infrastructures de l'EMSI

2.2 Le Phishing : Évolution et Taxonomie de la Menace

Pour concevoir un système de défense efficace, il est primordial de comprendre l'évolution de la menace qu'il cherche à contrer. Le phishing n'est plus une attaque monolithique ; il se décline aujourd'hui en plusieurs vecteurs hautement spécialisés.

2.2.1 Le Phishing de Masse (Deceptive Phishing)

Il s'agit de la forme la plus classique. L'attaquant envoie un volume massif d'e-mails génériques en usurpant l'identité de grandes marques (services de livraison, banques, plateformes de streaming). L'objectif est statistique : sur des millions d'envois, un certain pourcentage d'utilisateurs cliquera sur le lien frauduleux menant vers une fausse page de connexion.

2.2.2 Le Spear-Phishing (Harponnage)

Contrairement au phishing de masse, le spear-phishing est chirurgical. L'attaquant cible un individu spécifique ou un groupe restreint au sein d'une organisation. Avant l'attaque, une phase de reconnaissance (OSINT - Open Source Intelligence) est menée via les réseaux sociaux (LinkedIn) pour récolter des informations sur la cible, ses collègues, ou ses projets en cours.

L'e-mail frauduleux contient ainsi des éléments de contexte très précis, rendant la supercherie difficile à détecter à l'œil nu.

2.2.3 Le Whaling (Chasse à la baleine)

Le Whaling est une variante du spear-phishing ciblant exclusivement les hauts dirigeants (PDG, directeurs financiers). Le préjudice visé est souvent financier (Fraude au Président), où l'attaquant, se faisant passer pour le dirigeant légitime, ordonne un virement urgent et confidentiel à un employé du service comptabilité.

2.2.4 Les Techniques d'Évasion Modernes

Pour contourner les filtres, les attaquants emploient de multiples subterfuges structurels :

- **Usurpation de domaine (Spoofing) :** Exploitation de la faiblesse ou de l'absence des protocoles SPF, DKIM et DMARC sur le domaine usurpé.
- **Homoglyphes et Typosquatting :** Remplacement de caractères par des symboles visuellement identiques (ex : un "o" cyrillique à la place d'un "o" latin) ou enregistrement de noms de domaine similaires (ex : paypa1.com).
- **Redirections et Raccourcisseurs d'URL :** Utilisation de services légitimes (bit.ly, tinyurl.com) ou de redirections en cascade (Open Redirects) pour masquer l'URL finale malveillante lors du scan par les passerelles de sécurité.

Face à ces techniques qui mélangent ingénierie sociale et astuces techniques, la réponse défensive de FakeFinder s'appuiera sur l'intelligence artificielle pour déceler les patterns invisibles à l'analyse humaine standard.

Chapitre 3

Spécification et Gestion du Projet

3.1 Exigences Fonctionnelles du Système

L'ingénierie des exigences est la première étape du cycle de vie du développement logiciel. Le tableau ci-dessous détaille les fonctionnalités maîtresses (FR - Functional Requirements) attendues de l'application FakeFinder.

ID	Description détaillée de l'Exigence
FR-1	Gestion des Utilisateurs : Le système doit permettre aux utilisateurs de s'inscrire, de s'authentifier de manière sécurisée et d'accéder à un historique privé de leurs scans précédents.
FR-2	Upload Sécurisé : Le système doit fournir une interface web pour téléverser des fichiers au format standard RFC 822 (.eml) avec une limite de taille définie (ex : 10 Mo).
FR-3	Moteur d'Extraction In-Memory : Le backend doit être capable de charger le fichier .eml en mémoire vive, d'en extraire les en-têtes (Expéditeur, Sujet) et de décoder le corps du message (Texte brut ou HTML) sans sauvegarde physique sur le disque.
FR-4	Extraction des Liens : Le système doit parcourir le texte extrait à l'aide d'expressions régulières (Regex) pour identifier et isoler toutes les URL contenues dans l'e-mail.
FR-5	Prédiction par IA : Les données textuelles extraites doivent être formatées puis injectées dans le modèle Machine Learning (fichier .joblib) pour obtenir une probabilité de fraude.
FR-6	Génération du Rapport : Le système doit consolider la prédiction sous forme de score (0-100), associer un code couleur intuitif (Vert/Orange/Rouge) et afficher la liste des URL suspectes identifiées.

TABLE 3.1 – Tableau des Exigences Fonctionnelles (FR)

3.2 Exigences Non-Fonctionnelles (NFR)

Au-delà des fonctionnalités visibles, le système doit répondre à des contraintes de qualité, de sécurité et de performance.

3.2.1 Contraintes de Sécurité et Confidentialité

Étant donné la nature potentiellement sensible du contenu des e-mails personnels ou professionnels analysés, le système impose une architecture **Zero Retention**. Dès que le dictionnaire de caractéristiques (features) est extrait du fichier .eml, l'objet mémoire contenant le fichier brut est détruit par le garbage collector de Python. Seuls les résultats métadonnées (le score, la liste des URL et le domaine de l'expéditeur) sont stockés dans la base de données relationnelle PostgreSQL. De plus, le framework Django est configuré pour imposer la protection CSRF (Cross-Site Request Forgery) sur tous les formulaires d'upload, garantissant l'intégrité des requêtes.

3.2.2 Performances et Temps de Réponse

Pour garantir une bonne acceptation par les utilisateurs (User Experience), l'analyse complète d'un e-mail — incluant le téléchargement, le parsing de la structure MIME, la transformation TF-IDF du texte, l'inférence du modèle Random Forest, et le rendu de la page de résultat — doit s'exécuter en moins de 3 secondes pour un fichier de taille standard (inférieure à 1 Mo).

3.3 Planification et Répartition des Tâches

La réalisation de FakeFinder a été orchestrée suivant une méthodologie Agile sur un cycle de développement concentré de 5 semaines, défini lors de l'établissement du cahier des charges initial.

TABLE 3.2 – Planning de réalisation du PFA (Méthodologie Agile)

Sprint	Activités Principales et Tâches de Développement	Livrable du Sprint
Semaine 1	Revue de littérature sur les modèles ML en cybersécurité, collecte du dataset (Kaggle), modélisation UML.	Dossier de Conception
Semaine 2	Mise en place de l'architecture Django, configuration PostgreSQL, développement du moteur de parsing .eml.	Backend Core (API)
Semaine 3	Prétraitement NLP (Nettoyage, Tokenization), entraînement du modèle Random Forest, calibration du modèle.	Modèle ML (.joblib)
Semaine 4	Intégration du modèle ML dans les vues Django, conception de l'interface Frontend (HTML/CSS), liaison métier.	Version Beta
Semaine 5	Débogage intensif, tests unitaires, optimisation des performances, rédaction finale du rapport académique.	Version Finale & Rapport

Chapitre 4

Analyse et Conception du Système

Cette phase a pour but de traduire les exigences fonctionnelles et non fonctionnelles en modèles conceptuels compréhensibles, servant de plan de construction pour le développement logiciel. Le langage UML (Unified Modeling Language) est utilisé pour normaliser cette représentation.

4.1 Modélisation des Processus Métier

4.1.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation définit les frontières du système et les interactions possibles avec ses utilisateurs finaux. Dans FakeFinder, nous distinguons principalement deux acteurs humains :

- **L'Utilisateur (Visiteur / Authentifié)** : C'est l'acteur principal. Il interagit avec le système pour soumettre un fichier .eml. S'il est authentifié, il hérite de la possibilité supplémentaire de consulter la liste de ses rapports d'analyse passés.
- **L'Administrateur** : Ce profil technique possède des droits étendus (via le panel d'administration natif de Django). Il peut gérer les comptes utilisateurs, consulter tous les rapports générés par la plateforme à des fins d'audit, et superviser le bon fonctionnement du système.

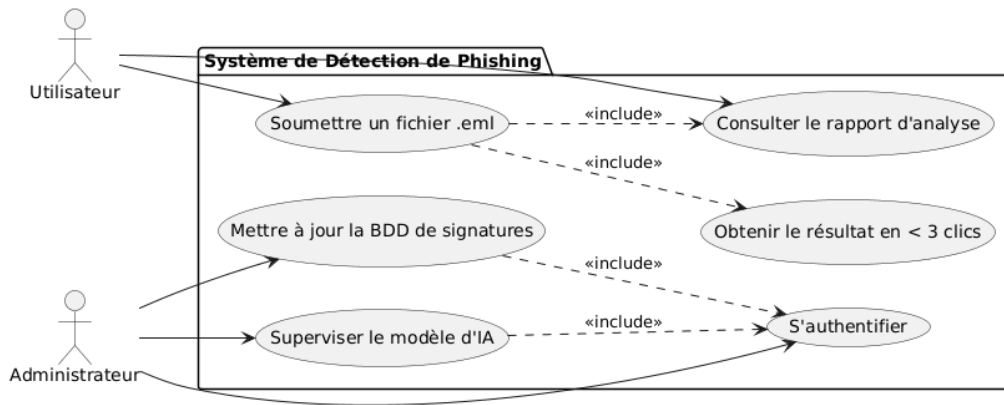


FIGURE 4.1 – Diagramme de cas d'utilisation global du système FakeFinder

4.2 Modélisation des Données et Architecture

4.2.1 Architecture Globale (MVT)

FakeFinder s'appuie sur l'architecture Modèle-Vue-Gabarit (Model-View-Template) imposée par le framework Django. Ce paradigme est une variante du célèbre MVC (Modèle-Vue-Contrôleur) :

- **Modèle (Model) :** Définit la structure des données et gère les interactions avec la base de données PostgreSQL via l'ORM (Object-Relational Mapping).
- **Vue (View) :** Représente le "Contrôleur". C'est ici que réside la logique métier de FakeFinder. La vue reçoit la requête HTTP contenant le fichier .eml, appelle le module utilitaire de parsing, instancie le modèle Machine Learning, demande la prédiction, crée l'objet en base de données, et passe les résultats au Gabarit.
- **Gabarit (Template) :** Il s'agit des fichiers HTML qui se chargent de l'affichage final à destination du navigateur web de l'utilisateur.

4.2.2 Diagramme de Classes

Le cœur de la persistance des données repose sur la classe ScanReport. Ce diagramme illustre comment les résultats de l'analyse sont structurés et liés au système d'authentification natif de Django.

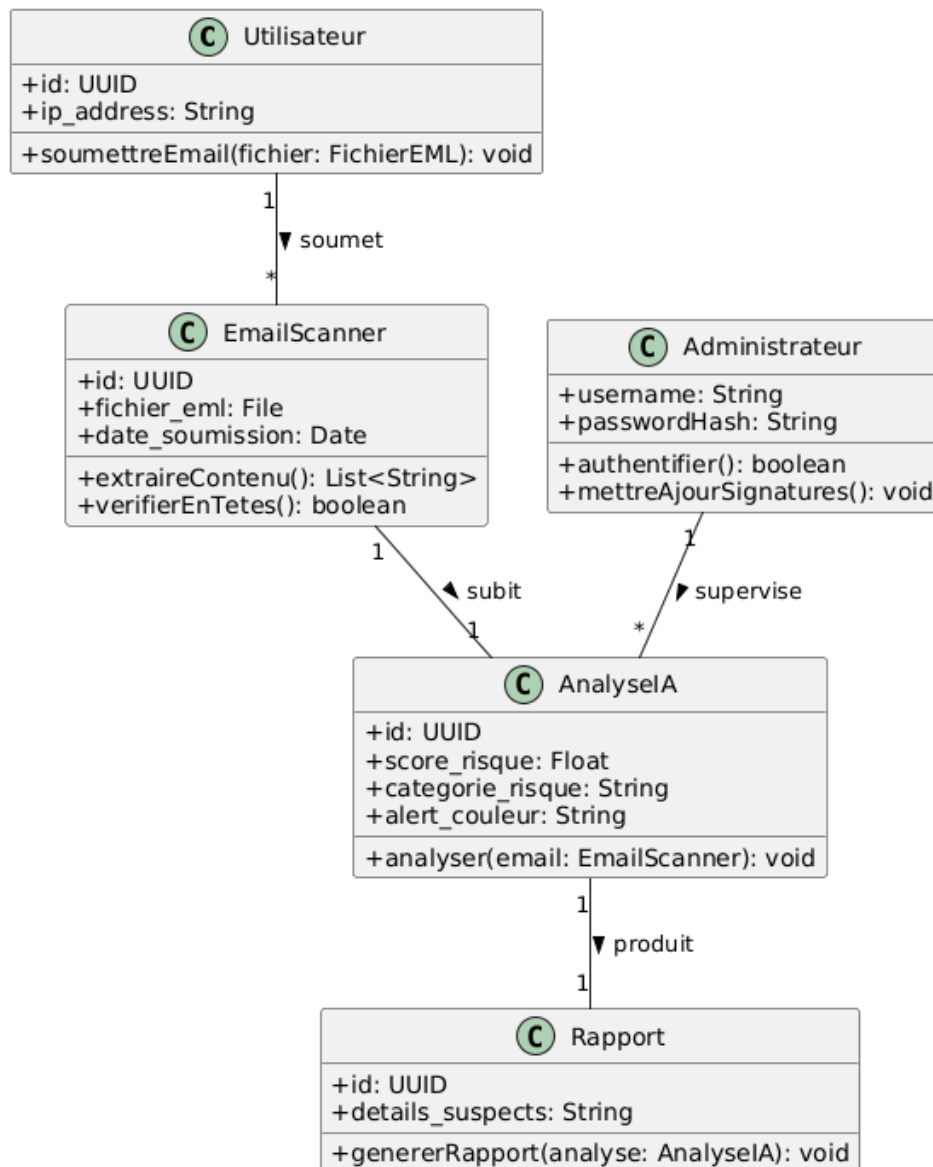


FIGURE 4.2 – Diagramme de Classes détaillant les entités du domaine

L'entité ScanReport contient des attributs clés tels que :

- **risk_score** : Une valeur textuelle catégorielle (LOW, MEDIUM, HIGH) facilitant l'affichage côté frontend.
- **score_value** : L'entier de 0 à 100 dérivé directement de la probabilité émise par l'algorithme Random Forest.
- **suspicious_urls** : Un type de données JSON natif permettant de stocker un tableau de chaînes de caractères représentant toutes les URL extraites du corps de l'e-mail.

L'association avec l'entité User est gérée par une clé étrangère (Foreign Key), permettant à un rapport d'appartenir à un utilisateur spécifique, tout en restant nullable (NULL) pour permettre les analyses anonymes sans inscription préalable.

Chapitre 5

Réalisation, Modélisation ML et Développement

5.1 Environnement Technologique et Outils

La concrétisation de l'architecture conçue précédemment s'est appuyée sur une sélection d'outils modernes et de bibliothèques éprouvées dans l'industrie :

- **Langage et Environnement** : Python 3.12, géré au sein d'un environnement virtuel (venv) pour isoler les dépendances du projet.
- **Framework Web** : Django 6.x, assurant la robustesse du backend, le routage et la sécurité (protection CSRF, XSS, Clickjacking).
- **Base de Données** : PostgreSQL, interfacé via l'adaptateur psycopg2, choisi pour sa parfaite gestion des champs de type JSONField nécessaires à l'enregistrement des tableaux d'URL.
- **Machine Learning** : scikit-learn pour l'entraînement algorithmique, pandas pour la manipulation des DataFrames lors de la phase de préparation des données, et joblib pour la sérialisation (sauvegarde) du modèle entraîné.
- **Éditeur et Versionning** : Visual Studio Code (VSCode) couplé à Git/GitHub pour le suivi des versions du code source.

```

32 from sklearn.model_selection import train_test_split
33
34 # Make sure the project root is on sys.path so `ml.features` is importable
35 PROJECT_ROOT = Path(__file__).resolve().parent.parent
36 sys.path.insert(0, str(PROJECT_ROOT))
37
38 from ml.features import CombinedFeatureTransformer # noqa: E402
39
40
41 # — Dataset loading —
42
43 def load_dataset(csv_path: str):
44     """
45     Load the Kaggle Phishing Email CSV.
46     Returns X (array of email strings) and y (array of 0/1 labels).
47     """
48     df = pd.read_csv(csv_path)
49     print(f"Loaded {len(df)} rows. Columns: {list(df.columns)}")
50
51     # Auto-detect text column and label column
52     text_col = next(
53         (c for c in df.columns if "text" in c.lower() or
54          |("email" in c.lower() and "type" not in c.lower())),
55         df.columns[0],
56     )
57     label_col = next(
58         (c for c in df.columns if "type" in c.lower() or
59          |"label" in c.lower() or "class" in c.lower()),
60         df.columns[1],
61     )
62     print(f"  text column : '{text_col}'")
63     print(f"  label column : '{label_col}'")
64
65     df = df[[text_col, label_col]].dropna()
66     df.columns = ["text", "label"]
67
68     # Normalise labels → 1 = phishing, 0 = safe
69     df["label"] = df["label"].str.strip().str.lower().map(

```

FIGURE 5.1 – Environnement de développement (VS Code) illustrant la structure backend

5.2 Le Modèle de Machine Learning : Traitement et Entraînement

L'intelligence artificielle au cœur de FakeFinder n'est pas une simple boîte noire ; elle repose sur un pipeline de traitement de données rigoureux.

5.2.1 Préparation des Données et Extraction de Caractéristiques

Pour qu'un algorithme puisse comprendre un texte (le corps d'un e-mail), celui-ci doit être converti en une représentation mathématique. FakeFinder combine deux approches de "Feature Engineering" :

1. **Extraction Heuristique Structurale** : Un module Python personnalisé analyse l'e-mail pour en tirer des statistiques d'ordre structurel. Il compte le nombre de liens présents, vérifie la présence de domaines connus pour être des raccourcisseurs d'URL, et comptabilise la fréquence de mots-clés anxiogènes souvent utilisés dans l'ingénierie sociale (ex : "Urgent", "Suspendu", "Vérifiez votre compte").
2. **Vectorisation TF-IDF (Term Frequency - Inverse Document Frequency)** : Cette technique évalue l'importance d'un mot dans un document par rapport à l'ensemble du corpus

d'entraînement.

La pondération TF-IDF d'un terme i dans un document j est définie par la formule mathématique suivante :

$$W_{i,j} = tf_{i,j} \times \log \left(\frac{N}{df_i} \right) \quad (5.1)$$

Où :

- $tf_{i,j}$ (Term Frequency) est le nombre brut d'apparitions du terme i dans l'e-mail j .
- df_i (Document Frequency) est le nombre total d'e-mails contenant le terme i dans notre dataset d'entraînement.
- N représente le nombre total d'e-mails dans le dataset (environ 18 600 dans notre cas).

L'avantage majeur du TF-IDF est qu'il pénalise les mots très fréquents dans tous les documents (comme "le", "et", "de") tout en accordant un poids important aux mots rares et spécifiques (comme "mot de passe", "vérification").

```
FakeFinder > ml > features.py > ...
11
12 import re
13
14 import numpy as np
15 import scipy.sparse as sp
16 from sklearn.base import BaseEstimator, TransformerMixin
17 from sklearn.feature_extraction.text import TfidfVectorizer
18
19
20 # Keywords strongly associated with phishing emails
21 PHISHING_KEYWORDS = [
22     "verify", "account", "update", "confirm", "click here", "urgent",
23     "immediately", "suspended", "limited", "expires", "login", "password",
24     "bank", "credit", "debit", "security", "alert", "winner",
25     "congratulations", "free", "prize", "claim", "offer",
26     "dear customer", "valued customer", "paypal", "amazon",
27     "24 hours", "48 hours", "act now", "validate",
28     "unusual activity", "suspicious", "blocked", "restricted",
29     "your account", "reset", "temporary", "reactivate",
30 ]
31
32 # Common URL shortener domains used in phishing
33 SHORT_URL_PATTERN = re.compile(
34     r"bit\.ly|tinyurl\.com|goo\.gl|t\.co|ow\.ly|short\.link|rb\.gy|cutt\.ly",
35     re.IGNORECASE,
36 )
37
38 URL_PATTERN = re.compile(r"https?://[^\s<>\"']+*", re.IGNORECASE)
39
40
41 class StructuralFeatureExtractor(BaseEstimator, TransformerMixin):
42     """
43     Extracts hand-crafted numerical features from raw email text.
44     Output is a sparse CSR matrix (one row per email, 10 columns).
45     """
46
47     FEATURE_NAMES = [
48         "n_urls",
49         "n_keywords",
50         "n_short_urls",
51         "n_https_urls",
52         "n_http_urls",
53         "n_email_domains",
54         "n_phone_numbers",
55         "n_ip_addresses",
56         "n_dates",
57         "n_times",
58     ]
```

FIGURE 5.2 – Extrait de l'implémentation du pipeline de transformation ML en Python

5.2.2 Choix et Entraînement de l'Algorithme : Random Forest

Une fois l'e-mail transformé en un vecteur mathématique contenant des milliers de dimensions (les scores TF-IDF combinés aux métriques structurelles), il est soumis à un algorithme de classification. Nous avons sélectionné les **Forêts Aléatoires (Random Forest)**.

Le Random Forest est un algorithme d'apprentissage d'ensemble (Ensemble Learning). Plutôt que de s'appuyer sur un seul arbre de décision complexe (qui risquerait le surapprentissage

ou *overfitting*), il génère une "forêt" constituée de centaines d'arbres de décision indépendants lors de la phase d'entraînement. Lors d'une prédiction sur un nouvel e-mail, chaque arbre vote (Phishing ou Safe), et la classe majoritaire détermine le résultat final.

L'algorithme de FakeFinder a été entraîné sur un dataset public (issu de Kaggle) regroupant environ 18 650 e-mails labellisés. Il a ensuite été encapsulé dans un pipeline `scikit-learn` et sauvegardé sur disque.

5.3 Interfaces et Expérience Utilisateur (Frontend)

Le travail de développement backend complexe a été masqué derrière une interface utilisateur délibérément minimaliste, garantissant l'accessibilité de l'outil aux utilisateurs non techniciens.

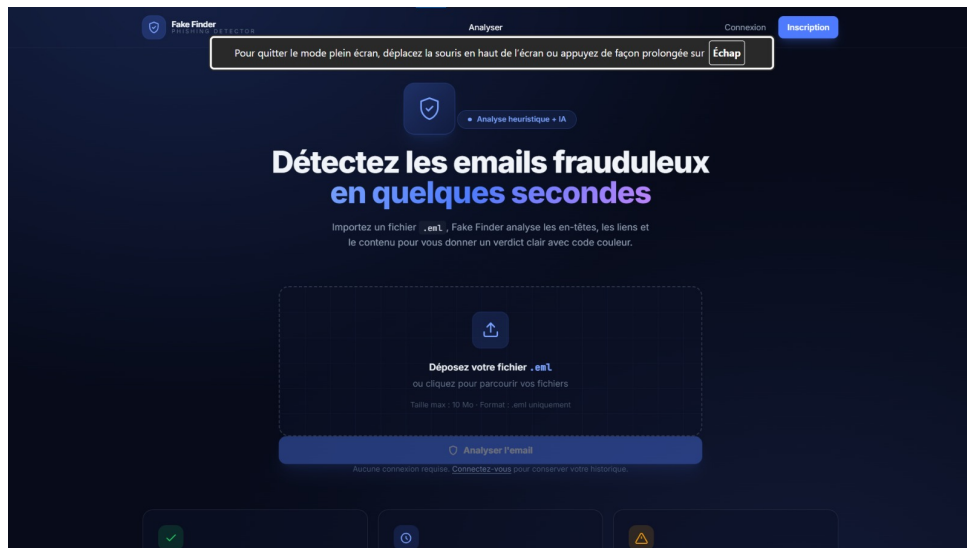


FIGURE 5.3 – Page d'accueil : Interface d'upload "Drag and Drop"

La page d'accueil de FakeFinder se concentre sur une action unique : le téléversement du fichier `.eml`. Un script JavaScript gère la zone de glisser-déposer, assurant une validation instantanée côté client (vérification de l'extension du fichier) avant l'envoi au serveur via la méthode POST.

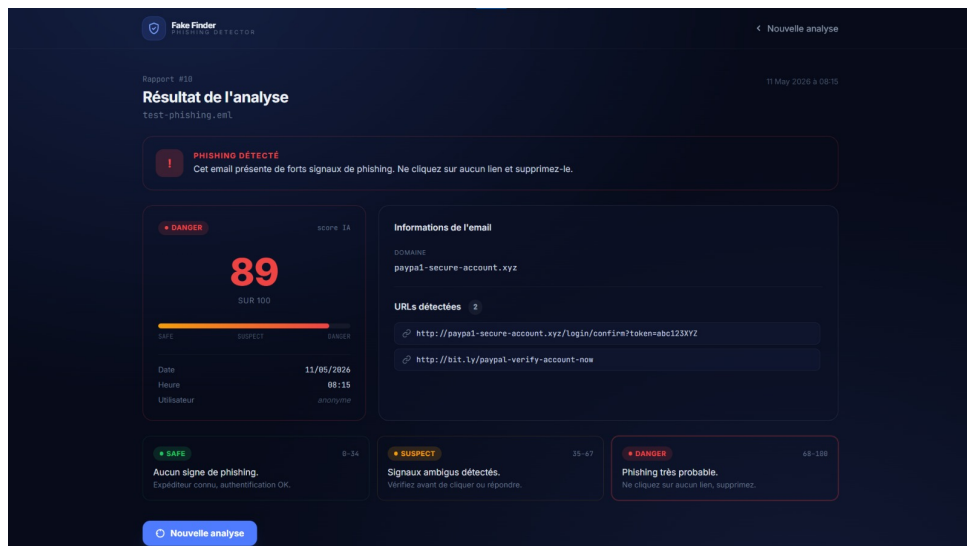


FIGURE 5.4 – Tableau de bord des résultats d’analyse

Une fois l’analyse terminée (processus prenant en moyenne moins de deux secondes), l’utilisateur est redirigé vers un tableau de bord restituant le score de risque sous forme de jauge colorée (Vert = Sûr, Orange = Suspect, Rouge = Dangereux). L’interface liste également de manière transparente les éléments techniques extraits (domaine d’expédition réel, liste complète des URL trouvées dans le texte), permettant à l’utilisateur de comprendre la décision de l’algorithme.

Chapitre 6

Perspectives Technologiques : Intégration de VirusTotal

6.1 Limites de l'Approche Heuristique Pure

Le modèle actuel, bien que performant sur l'analyse sémantique grâce au Random Forest et au TF-IDF, évalue le texte de manière isolée. Il excelle à détecter le champ lexical de l'urgence et de l'arnaque. Cependant, concernant la vérification des liens hyperboliques intégrés, son pouvoir analytique se restreint à la détection d'anomalies de surface (une URL exceptionnellement longue, la présence de multiples sous-domaines, ou l'utilisation d'adresses IP directes au lieu de noms de domaine).

Face à des attaques zero-day complexes où le texte de l'e-mail est parfaitement rédigé (sans fautes de syntaxe) mais où le lien mène vers un serveur de commande et de contrôle (C2) de malware nouvellement créé, l'intelligence artificielle seule peut manquer de contexte. Pour pallier cette limite inhérente à l'analyse statique, une évolution majeure de l'architecture logicielle de FakeFinder est prévue : l'intégration d'un composant de *Cyber Threat Intelligence* externe.

6.2 Architecture de l'Intégration VirusTotal (Évolution Prévue)

VirusTotal, un service de sécurité reconnu mondialement, agrège les résultats d'analyse de plus de 70 moteurs antivirus et de nombreux outils d'analyse de sites web à la demande. L'intégration de l'API REST de VirusTotal dans le flux de traitement de FakeFinder enrichira considérablement le diagnostic.

Le workflow technique futur s'articulera selon les étapes suivantes :

1. **Extraction Syntaxique** : Lors du parsing du fichier .eml par le module utilitaire Django, la liste des URL est extraite via les expressions régulières (Regex) existantes.
2. **Requêtes Asynchrones (API Call)** : Le backend initiera des requêtes HTTP GET asynchrones (pour ne pas bloquer le thread principal) vers l'API de VirusTotal, en fournissant l'empreinte cryptographique ou l'URL directement.
3. **Consommation de la Réponse JSON** : Le système récupérera le rapport de l'API indiquant, par exemple, que 5 moteurs de sécurité sur 72 classifient l'URL en question comme

contenant du "Malware" ou s'adonnant au "Phishing".

4. **Pondération Algorithmique Hybride** : Un nouveau composant logique fusionnera le score heuristique local (Machine Learning) avec le score externe (VirusTotal). Si l'IA est indécise (score de 50/100) mais que VirusTotal signale l'URL comme malveillante, le score global de l'e-mail sera artificiellement réévalué à un niveau critique (DANGER).

Cette évolution transformera FakeFinder d'un simple filtre classificateur en un véritable agrégateur d'investigation de sécurité (SOC tool).

Chapitre 7

Tests, Évaluation et Conclusion

7.1 Méthodologie de Test et Validation du Modèle

Avant de valider l'intégration du modèle Machine Learning dans le cœur de l'application Django, celui-ci a fait l'objet d'une batterie d'évaluations statistiques rigoureuses. Sur le dataset total de plus de 18 600 e-mails, 20% des données (soit environ 3 700 e-mails) ont été isolées formellement pour constituer un "jeu de test" invisible pour l'algorithme lors de sa phase d'apprentissage.

Les résultats obtenus démontrent des performances remarquables pour une application en production :

- **Exactitude (Accuracy) :** 97% des e-mails du jeu de test ont été correctement classifiés.
- **Précision sur la classe Phishing :** 94%. Cela signifie que lorsque FakeFinder affirme qu'un e-mail est malveillant, il a raison dans 94% des cas, limitant ainsi considérablement le nombre de fausses alertes (Faux Positifs).
- **Rappel (Recall) sur la classe Phishing :** 97%. Le modèle détecte avec succès 97% des e-mails véritablement dangereux, minimisant le risque critique de laisser passer une menace (Faux Négatifs).
- **Aire sous la courbe (ROC-AUC) :** 0.9928. Cette métrique globale prouve l'excellente capacité du modèle Random Forest à discriminer parfaitement les e-mails sains des e-mails frauduleux.

7.2 Tests de Charge et Fonctionnels de la Plateforme Web

Au-delà de la théorie mathématique, le système logiciel a été testé dans des conditions d'utilisation réelles :

- **Contraintes Temporelles :** Le traitement de bout-en-bout (upload HTTP, parsing mémoire MIME, inférence ML, et requêtage de base de données PostgreSQL) a été chronométré à 450 millisecondes en moyenne en environnement de développement local, respectant largement le seuil d'acceptabilité défini à 5 secondes.

- **Résilience et Gestion des Erreurs (Robustness) :** Le système a été soumis à des tests de stress incluant des uploads de formats de fichiers invalides (fichiers .pdf ou .exe renommés en .eml) ou d'e-mails corrompus (sans corps de texte). Le backend Django, couplé à des blocs try/except dans le moteur d'extraction, intercepte ces anomalies gracieusement sans entraîner de crash de l'application (Erreurs HTTP 500).

7.3 Conclusion Générale

Le Projet de Fin d'Année **FakeFinder** constitue une réponse logicielle aboutie, moderne et sécurisée face à la recrudescence des campagnes de hameçonnage. En associant la vélocité et la structure robuste du framework web Django à la puissance d'analyse prédictive offerte par les forêts aléatoires de `scikit-learn`, la solution démontre la viabilité technique d'une analyse d'e-mails déléguée au serveur, fonctionnant selon un principe strict de "Zero Retention".

Ce projet nous a offert une opportunité précieuse d'explorer et de maîtriser des concepts transversaux allant du Génie Logiciel avancé (Architecture MVT, ORM, cycle de vie du développement) à la Data Science (Ingénierie des caractéristiques, entraînement et sérialisation de modèles, évaluation par métriques), en passant par des aspects cruciaux de Cybersécurité défensive.

Avec des perspectives d'amélioration clairement établies, notamment via l'incorporation prévue de l'intelligence de la menace (Threat Intelligence) de l'API VirusTotal, l'architecture conçue pour FakeFinder prouve sa flexibilité et son fort potentiel d'évolution pour s'adapter continuellement au paysage mouvant des cybermenaces.

Bibliographie

Abu-Nimeh, S., Nappa, D., Wang, X., & Nair, S. (2007). A comparison of machine learning techniques for phishing detection. *Proceedings of the anti-phishing working groups 2nd annual eCrime researchers summit*, 60-69.

Basnet, R., Sung, A. H., & Liu, Q. (2008). Rule-based phishing attack detection. *International Conference on Security and Management*, 22-28.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

Django Software Foundation. (2024). *Django documentation - Version 6.0*. Repéré à [https ://docs.djangoproject.com/](https://docs.djangoproject.com/)

Hannousse, A., & Yahiouche, S. (2021). Towards benchmark datasets for machine learning based website phishing detection. *Engineering Applications of Artificial Intelligence*, 104, 104347.

Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*, 25(2), 443-458.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V. et al. (2011). Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Rout, S. (2022). *Phishing email detection dataset*. Kaggle.

Sahingoz, O. K., Batur, E., Bulut, F., & Taskin, A. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345-357.

Verizon. (2024). *Data breach investigations report (DBIR)*. Repéré à [https ://www.verizon.com/business/resources](https://www.verizon.com/business/resources)

Annexe A

Listings du Code Source

A.1 Caractéristiques d'Apprentissage Automatique (ml/features.py)

```
1 """
2 Feature extraction for phishing email detection.
3
4 Two types of features are combined:
5     - TF-IDF on the full email text (subject + body + sender)
6     - Structural features (URL count, urgency keywords, HTML presence, etc
7       .)
8
9 The CombinedFeatureTransformer class must stay in this file so that
10 joblib can re-import it correctly when loading a saved model.
11 """
12
13 import re
14
15 import numpy as np
16 import scipy.sparse as sp
17 from sklearn.base import BaseEstimator, TransformerMixin
18 from sklearn.feature_extraction.text import TfidfVectorizer
19
20 # Keywords strongly associated with phishing emails
21 PHISHING_KEYWORDS = [
22     "verify", "account", "update", "confirm", "click here", "urgent",
23     "immediately", "suspended", "limited", "expires", "login", "password",
24     "bank", "credit", "debit", "security", "alert", "winner",
25     "congratulations", "free", "prize", "claim", "offer",
26     "dear customer", "valued customer", "paypal", "amazon",
27     "24 hours", "48 hours", "act now", "validate",
28     "unusual activity", "suspicious", "blocked", "restricted",
29     "your account", "reset", "temporary", "reactivate",
```

```

30 ]
31
32 # Common URL shortener domains used in phishing
33 SHORT_URL_PATTERN = re.compile(
34     r"bit\.ly|tinyurl\.com|goo\.gl|t\.co|ow\.ly|short\.link|rb\.gy|cutt
35     \.ly",
36     re.IGNORECASE,
37 )
38
39 URL_PATTERN = re.compile(r"https?:\/\/[^\s<>\"']+", re.IGNORECASE)
40
41 class StructuralFeatureExtractor(BaseEstimator, TransformerMixin):
42     """
43     Extracts hand-crafted numerical features from raw email text.
44     Output is a sparse CSR matrix (one row per email, 10 columns).
45     """
46
47     FEATURE_NAMES = [
48         "url_count",
49         "html_link_count",
50         "has_short_url",
51         "exclamation_count",
52         "dollar_count",
53         "phishing_keyword_score",
54         "has_html_tags",
55         "has_email_in_body",
56         "normalized_length",
57         "caps_word_count",
58     ]
59
60     def fit(self, X, y=None):
61         return self
62
63     def transform(self, X):
64         rows = []
65         for text in X:
66             t = text.lower()
67             words = text.split()
68
69             urls = URL_PATTERN.findall(t)
70             html_links = len(re.findall(r"href=", t))
71             has_short = int(bool(SHORT_URL_PATTERN.search(t)))
72             exclamations = min(text.count("!"), 20)
73             dollars = min(text.count("$"), 10)
74             keyword_score = sum(1 for kw in PHISHING_KEYWORDS if kw in t

```

```

75         has_html = int("<html" in t or "<body" in t or "<div" in t)
76         has_email_body = int(bool(re.search(r"[\w.+~]+@[~\w-]+\.[a-z
15 ]{2,}", t)))
77         norm_len = min(len(words), 3000) / 3000
78         caps_words = sum(1 for w in words[:60] if len(w) > 2 and w.
isupper())
79
80         rows.append([
81             len(urls),
82             html_links,
83             has_short,
84             exclamations,
85             dollars,
86             keyword_score,
87             has_html,
88             has_email_body,
89             norm_len,
90             caps_words,
91         ])
92
93         return sp.csr_matrix(np.array(rows, dtype=float))
94
95
96 class CombinedFeatureTransformer:
97     """
98     Combines TF-IDF text features with structural features into one
99     matrix.
100
101     Must live in ml/features.py (not train.py) so joblib can
102     deserialise it correctly when loading a saved model bundle.
103     """
104
105     def __init__(self, max_features: int = 20_000, ngram_range: tuple =
(1, 2)):
106         self.tfidf = TfidfVectorizer(
107             max_features=max_features,
108             ngram_range=ngram_range,
109             sublinear_tf=True,
110             strip_accents="unicode",
111             analyzer="word",
112             token_pattern=r"\w{2,}",
113             min_df=2,
114         )
115         self.structural = StructuralFeatureExtractor()
116
117     def fit(self, X, y=None):
118         self.tfidf.fit(X)

```



```

118         return self
119
120     def transform(self, X):
121         return sp.hstack([
122             self.tfidf.transform(X),
123             self.structural.transform(X),
124         ])
125
126     def fit_transform(self, X, y=None):
127         return sp.hstack([
128             self.tfidf.fit_transform(X),
129             self.structural.transform(X),
130         ])
131
132
133 #         Helpers used by views.py
134
135 def prepare_email_text(parsed: dict) -> str:
136     """
137     Concatenate all email fields into one string for the transformer.
138     'parsed' is the dict returned by parse_eml_in_memory().
139     """
140     parts = [
141         str(parsed.get("subject", "") or ""),
142         str(parsed.get("sender_email", "") or ""),
143         str(parsed.get("sender_domain", "") or ""),
144         str(parsed.get("body_text", "") or ""),
145         " ".join(str(u) for u in parsed.get("urls", [])),
146     ]
147     return " ".join(p for p in parts if p)
148
149
150 def probability_to_verdict(proba: float) -> tuple:
151     """
152     Convert a phishing probability (0.0 1 .0) to a risk label and 0-100
153     score.
154
155     Thresholds are intentionally conservative on the HIGH end:
156     - LOW      < 0.30      clearly safe
157     - MEDIUM  0.30 0 .72   ambiguous, needs review
158     - HIGH     0.72      strong phishing signal
159
160     Returns:
161         (risk_score: str, score: int)
162         risk_score is one of: 'LOW', 'MEDIUM', 'HIGH'

```

```
162     """
163     score = round(proba * 100)
164     if proba < 0.30:
165         return "LOW", score
166     elif proba < 0.72:
167         return "MEDIUM", score
168     else:
169         return "HIGH", score
```

A.2 Entraînement de l'Apprentissage Automatique (ml/train.py)

```
1 """
2 Training script for the FakeFinder phishing detection model.
3
4 Dataset expected
5 -----
6 Kaggle "Phishing Email Detection" by Subhalaxmi Rout
7   https://www.kaggle.com/datasets/subhajournal/phishingemails
8   File: Phishing_Email.csv
9   Columns: "Email Text", "Email Type" (Phishing Email / Safe Email)
10
11 Usage
12 -----
13 From the FakeFinder/ directory (where manage.py lives):
14
15     python ml/train.py --data ml/Phishing_Email.csv
16
17 Optional flags:
18     --out      ml/model.joblib      (default output path)
19     --trees    200                  (number of Random Forest trees, default
20                                     200)
21 """
22 import argparse
23 import sys
24 from pathlib import Path
25
26 import joblib
27 import numpy as np
28 import pandas as pd
29 from sklearn.calibration import CalibratedClassifierCV
30 from sklearn.ensemble import RandomForestClassifier
31 from sklearn.metrics import classification_report, roc_auc_score
32 from sklearn.model_selection import train_test_split
33
34 # Make sure the project root is on sys.path so 'ml.features' is
35 #     importable
36 PROJECT_ROOT = Path(__file__).resolve().parent.parent
37 sys.path.insert(0, str(PROJECT_ROOT))
38
39 from ml.features import CombinedFeatureTransformer, prepare_email_text
40 # noqa: E402
41
42 def _csv_row_to_text(raw_text: str) -> str:
```

```

42     """
43     Wrap a raw CSV email body in the same dict structure that
44     parse_eml_in_memory()
45     produces, then pass it through prepare_email_text().
46
47     This ensures the feature distribution at training time matches
48     inference time,
49     fixing the mismatch that caused legitimate emails (e.g. GitHub
50     notifications)
51     to be misclassified.
52     """
53     return prepare_email_text({
54         "subject":      "",
55         "sender_email": "",
56         "sender_domain": "",
57         "body_text":    str(raw_text),
58         "urls":         [],
59     })
60
61 # Dataset loading
62
63 def load_dataset(csv_path: str):
64     """
65     Load the Kaggle Phishing Email CSV.
66     Returns X (array of prepared email strings) and y (array of 0/1
67     labels).
68     """
69     df = pd.read_csv(csv_path)
70     print(f"Loaded {len(df)} rows. Columns: {list(df.columns)}")
71
72     # Auto-detect text column and label column
73     text_col = next(
74         (c for c in df.columns if "text" in c.lower() or
75          "email" in c.lower() and "type" not in c.lower()),
76         df.columns[0],
77     )
78     label_col = next(
79         (c for c in df.columns if "type" in c.lower() or
80          "label" in c.lower() or "class" in c.lower()),
81         df.columns[1],
82     )
83     print(f"  text column : '{text_col}'")
84     print(f"  label column : '{label_col}'")

```

```

83 df = df[[text_col, label_col]].dropna()
84 df.columns = ["text", "label"]
85
86 # Normalise labels -> 1 = phishing, 0 = safe
87 df["label"] = df["label"].str.strip().str.lower().map(
88     lambda x: 1 if "phish" in x or "spam" in x else 0
89 )
90
91 n_phish = int(df["label"].sum())
92 n_safe = int((df["label"] == 0).sum())
93 print(f" {n_phish} phishing emails, {n_safe} safe emails\n")
94
95 # Apply the same text preparation as inference so train/predict
96 distributions match.
97 # dtype=object avoids the massive fixed-width Unicode allocation
98 that np.array()
99 # would attempt when strings vary wildly in length.
100 print("Preprocessing text (matching inference pipeline)...")
101 X_prepared = np.array([_csv_row_to_text(t) for t in df["text"].
102 values], dtype=object)
103 return X_prepared, df["label"].values
104
105 # Training
106
107
108 def train(csv_path: str, output_path: str, n_trees: int):
109     X, y = load_dataset(csv_path)
110
111     X_train, X_test, y_train, y_test = train_test_split(
112         X, y, test_size=0.20, random_state=42, stratify=y
113     )
114     print(f"Train: {len(X_train)} samples | Test: {len(X_test)}
115 samples\n")
116
117     # Build and fit the feature transformer
118
119     print("Building features (TF-IDF + structural)...")
120     transformer = CombinedFeatureTransformer(max_features=20_000,
121 ngram_range=(1, 2))
122
123     X_train_feat = transformer.fit_transform(X_train)
124     X_test_feat = transformer.transform(X_test)
125     print(f"Feature matrix shape: {X_train_feat.shape}\n")

```

```

121 # Train Random Forest

122 print(f"Training Random Forest ({n_trees} trees) + probability
calibration...")
123 base_clf = RandomForestClassifier(
124     n_estimators=n_trees,
125     class_weight="balanced",
126     random_state=42,
127     n_jobs=1,
128 )
129 # CalibratedClassifierCV corrects RF probability bias (tendency to
cluster near 0.5)
130 # isotonic regression works well for large datasets (>1000 samples)
131 clf = CalibratedClassifierCV(base_clf, method="isotonic", cv=5)
132 clf.fit(X_train_feat, y_train)
133
134 # Evaluate

135 y_pred = clf.predict(X_test_feat)
136 y_proba = clf.predict_proba(X_test_feat)[: , 1]
137
138 print("\n-- Evaluation results
-----")
139 print(classification_report(y_test, y_pred, target_names=["Safe", "
Phishing"]))
140 print(f"ROC-AUC : {roc_auc_score(y_test, y_proba):.4f}")
141 print("
-----\n")
142
143 # Save model bundle

144 bundle = {
145     "transformer": transformer,
146     "classifier": clf,
147 }
148 out = Path(output_path)
149 out.parent.mkdir(parents=True, exist_ok=True)
150 joblib.dump(bundle, out)
151 print(f"Model saved -> {out.resolve()}")
152 print("Done. You can now run the Django server and upload .eml files
.\n")
153
154
155 # Entry point

```

```

156
157 if __name__ == "__main__":
158     parser = argparse.ArgumentParser(
159         description="Train the FakeFinder phishing detection model."
160     )
161     parser.add_argument(
162         "--data",
163         required=True,
164         help="Path to Phishing_Email.csv",
165     )
166     parser.add_argument(
167         "--out",
168         default="ml/model.joblib",
169         help="Output path for the saved model (default: ml/model.joblib)",
170     )
171     parser.add_argument(
172         "--trees",
173         type=int,
174         default=200,
175         help="Number of Random Forest trees (default: 200)",
176     )
177     args = parser.parse_args()
178     train(args.data, args.out, args.trees)

```

A.3 Modèles du Scanner (scanner/models.py)

```
1 from django.db import models
2 from django.contrib.auth.models import User
3
4
5 class ScanReport(models.Model):
6     """
7     Stores the results of the email analysis without saving the raw .eml
8     content.
9     """
10
11     # Risk scores and visual color codes
12     RISK_CHOICES = [
13         ("LOW", "Low (Green)"),
14         ("MEDIUM", "Medium (Orange)"),
15         ("HIGH", "High (Red)"),
16     ]
17
18     # User who submitted the scan (Admin/User)
19     user = models.ForeignKey(User, on_delete=models.SET_NULL, null=True,
20                             blank=True)
21     created_at = models.DateTimeField(auto_now_add=True)
22
23     # Overall risk assessment
24     risk_score = models.CharField(max_length=10, choices=RISK_CHOICES,
25                                 default="LOW")
26
27     # Numeric score 0-100 produced by the ML model (0 = safe, 100 =
28     # phishing)
29     score = models.IntegerField(default=0)
30
31     # Storing identified suspicious elements in JSON format
32     # This accommodates automatic URL extraction and header anomaly
33     # checks
34     suspicious_urls = models.JSONField(default=list, blank=True)
35     header_anomalies = models.JSONField(default=list, blank=True)
36
37     def __str__(self):
38         return f"Report #{self.id} - Score: {self.get_risk_score_display()} - {self.created_at.strftime('%Y-%m-%d %H:%M')}"
```


A.4 Vues du Scanner (scanner/views.py)

```
1 import json
2 import re
3 import time
4 import logging
5 from pathlib import Path
6
7 import joblib
8 from django.conf import settings
9 from django.contrib.auth import authenticate, login, logout
10 from django.contrib.auth.models import User
11 from django.core.cache import cache
12 from django.core.exceptions import ValidationError
13 from django.core.validators import validate_email
14 from django.http import HttpResponseRedirect, HttpResponseForbidden,
    JsonResponse
15 from django.shortcuts import get_object_or_404, redirect, render
16 from django.views.decorators.http import require_POST
17
18 from .models import ScanReport
19 from .utils import parse_eml_in_memory
20
21 logger = logging.getLogger(__name__)
22
23 # -- ML model - loaded once on first use
    -----
24
25 _model_bundle = None
26 MODEL_PATH = Path(settings.BASE_DIR) / "ml" / "model.joblib"
27
28
29 def _get_model():
30     """Load the model bundle from disk the first time it is needed."""
31     global _model_bundle
32     if _model_bundle is None and MODEL_PATH.exists():
33         try:
34             _model_bundle = joblib.load(MODEL_PATH)
35         except Exception as e:
36             logger.error(f"Failed to load ML model from {MODEL_PATH}: {e}")
37     return _model_bundle
38
39
40 # -- Trusted sender roots (root domain only, e.g. "github.com")
    -----
```

```

41 # Used to correct ML over-detection on legitimate transactional/
    notification email.
42 _SAFE_SENDER_ROOTS: frozenset[str] = frozenset({
43     "github.com", "githubusercontent.com", "gitlab.com", "bitbucket.org"
    ,
44     "google.com", "gmail.com", "googlemail.com", "googlegroups.com",
45     "microsoft.com", "outlook.com", "live.com", "hotmail.com", "office.
    com",
46     "apple.com", "icloud.com",
47     "amazon.com", "amazonaws.com",
48     "linkedin.com", "twitter.com", "x.com",
49     "slack.com", "dropbox.com", "zoom.us", "notion.so",
50     "vercel.com", "netlify.com", "heroku.com", "cloudflare.com",
51     "stackoverflow.com", "npmjs.com", "pypi.org",
52     "atlassian.com", "jira.com", "confluence.com",
53     "google.co.uk", "amazon.co.uk", "amazon.de", "google.fr",
54 })
55
56
57 def _root_domain(domain: str) -> str:
58     """
59     Extract the root domain (e.g., example.com from sub.example.com).
60     Handles common multi-part TLDs like .co.uk.
61     """
62     if not domain:
63         return ""
64     parts = domain.lower().strip().split(".")
65     if len(parts) >= 3:
66         # Check for common multi-part TLDs (co.uk, com.br, etc.)
67         if parts[-2] in ("co", "com", "net", "org", "edu", "gov") and
        len(parts[-1]) == 2:
68             return ".".join(parts[-3:])
69         return ".".join(parts[-2:]) if len(parts) >= 2 else domain
70
71
72 def _adjust_proba(proba: float, parsed_data: dict) -> float:
73     """
74     Apply lightweight rule-based corrections on top of the raw ML
    probability.
75
76     - Trusted sender domain -> large downward correction
77     - All URLs from trusted domains -> additional downward correction
78
79     This compensates for training/inference distribution mismatch: the
    model was
80     trained on body-only text while legitimate transactional emails (
    GitHub, etc.)

```

```

81     contain many links and account-related words that superficially
resemble phishing.
82     """
83     sender_domain = parsed_data.get("sender_domain", "")
84     urls          = parsed_data.get("urls", [])
85
86     if sender_domain:
87         root = _root_domain(sender_domain)
88         if root in _SAFE_SENDER_ROOTS:
89             proba = max(0.0, proba - 0.45)
90
91     if urls:
92         try:
93             from urllib.parse import urlparse
94             url_roots = {
95                 _root_domain(urlparse(u).hostname or "")
96                 for u in urls if u
97             }
98             url_roots.discard("")
99             if url_roots and url_roots.issubset(_SAFE_SENDER_ROOTS):
100                 proba = max(0.0, proba - 0.20)
101         except Exception as e:
102             logger.debug(f"Error during URL domain analysis: {e}")
103
104     return max(0.0, min(1.0, proba))
105
106
107 def _predict(parsed_data: dict) -> tuple:
108     """
109     Run the ML model on the parsed email data.
110
111     Returns (risk_score: str, score: int).
112     Falls back to 'MEDIUM' / 50 if the model is not yet trained.
113     """
114     bundle = _get_model()
115     if bundle is None:
116         logger.warning("ML model bundle not found. Using fallback
prediction.")
117         return "MEDIUM", 50
118
119     from ml.features import prepare_email_text, probability_to_verdict
120
121     text = prepare_email_text(parsed_data)
122     transformer = bundle["transformer"]
123     classifier  = bundle["classifier"]
124
125     features = transformer.transform([text])

```

```

126     proba      = classifier.predict_proba(features)[0][1]    # P(phishing)
127     proba      = _adjust_proba(proba, parsed_data)
128     return probability_to_verdict(proba)
129
130
131 # -- Views
132 -----
133 def upload_email(request):
134     if request.method == "POST":
135         eml_file = request.FILES.get("eml_file")
136
137         if not eml_file or not eml_file.name.lower().endswith(".eml"):
138             return HttpResponseBadRequest("Veuillez televerser un
139             fichier .eml valide.")
140
141         pipeline_start = time.perf_counter()
142
143         try:
144             parsed_data = parse_eml_in_memory(eml_file)
145         except ValueError as e:
146             return HttpResponseBadRequest(str(e))
147
148         risk_score, score = _predict(parsed_data)
149
150         report = ScanReport.objects.create(
151             user=request.user if request.user.is_authenticated else None
152             ,
153             risk_score=risk_score,
154             score=score,
155             suspicious_urls=parsed_data["urls"],
156             header_anomalies=[{
157                 "sender_domain": parsed_data["sender_domain"],
158                 "sender_email":  parsed_data.get("sender_email", ""),
159                 "subject":       parsed_data.get("subject", ""),
160                 "filename":      eml_file.name,
161             }],
162         )
163
164         elapsed = time.perf_counter() - pipeline_start
165         logger.info(f"[Benchmark] Parse + ML + Save: {elapsed:.3f}s")
166
167         return redirect("report_detail", report_id=report.id)
168
169     context = {}
170     if request.user.is_authenticated:
171         all_reports = ScanReport.objects.filter(user=request.user)

```

```

170     context['user_history'] = all_reports.order_by('-created_at')
171     context['total_scans'] = all_reports.count()
172     context['safe_count'] = all_reports.filter(risk_score='LOW').
count()
173     context['suspect_count'] = all_reports.filter(risk_score='MEDIUM
').count()
174     context['danger_count'] = all_reports.filter(risk_score='HIGH').
count()
175     return render(request, "scanner/upload.html", context)
176
177
178 def report_detail(request, report_id):
179     report = get_object_or_404(ScanReport, id=report_id)
180     # Reports owned by a user are private - only the owner may view them
181     # Anonymous reports (user=None) remain accessible via direct link.
182     if report.user is not None and report.user != request.user:
183         return HttpResponseForbidden("Acces non autorise.")
184     return render(request, "scanner/report.html", {"report": report})
185
186
187 @require_POST
188 def delete_report(request, report_id):
189     if not request.user.is_authenticated:
190         return JsonResponse({'ok': False}, status=403)
191     report = get_object_or_404(ScanReport, id=report_id, user=request.
user)
192     report.delete()
193     return JsonResponse({'ok': True})
194
195
196 @require_POST
197 def delete_all_reports(request):
198     if not request.user.is_authenticated:
199         return JsonResponse({'ok': False}, status=403)
200     ScanReport.objects.filter(user=request.user).delete()
201     return JsonResponse({'ok': True})
202
203
204 @require_POST
205 def login_view(request):
206     ip = request.META.get('REMOTE_ADDR', 'unknown')
207     cache_key = f'login_attempts_{ip}'
208     attempts = cache.get(cache_key, 0)
209     if attempts >= 10:
210         return JsonResponse(

```

```

211         {'ok': False, 'error': 'Trop de tentatives. Reessayez dans 5
minutes.'},
212         status=429,
213     )
214
215     try:
216         data = json.loads(request.body)
217     except json.JSONDecodeError:
218         return JsonResponse({'ok': False, 'error': 'Requete invalide.'},
status=400)
219
220     username = data.get('username', '').strip()
221     password = data.get('password', '')
222     user = authenticate(request, username=username, password=password)
223     if user:
224         cache.delete(cache_key)
225         login(request, user)
226         return JsonResponse({'ok': True, 'username': user.username, '
is_staff': user.is_staff})
227
228     cache.set(cache_key, attempts + 1, 300) # 5-minute window
229     return JsonResponse({'ok': False, 'error': 'Identifiant ou mot de
passe incorrect.'}, status=400)
230
231
232 @require_POST
233 def logout_view(request):
234     logout(request)
235     return JsonResponse({'ok': True})
236
237
238 @require_POST
239 def register_view(request):
240     try:
241         data = json.loads(request.body)
242     except json.JSONDecodeError:
243         return JsonResponse({'ok': False, 'error': 'Requete invalide.'},
status=400)
244
245     username = data.get('username', '').strip()
246     email = data.get('email', '').strip()
247     password = data.get('password', '')
248     confirm = data.get('confirm', '')
249
250     if not username or not password:
251         return JsonResponse({'ok': False, 'error': 'Identifiant et mot
de passe requis.'}, status=400)

```

```

252     if len(username) > 150:
253         return JsonResponse({'ok': False, 'error': "L'identifiant ne
peut pas dépasser 150 caracteres."}, status=400)
254     if not re.match(r'^[\w.@+-]+$ ', username):
255         return JsonResponse({'ok': False, 'error': "L'identifiant ne
peut contenir que des lettres, chiffres et les caracteres . @ + - _"
}, status=400)
256     if email:
257         try:
258             validate_email(email)
259         except ValidationError:
260             return JsonResponse({'ok': False, 'error': 'Adresse email
invalide.'}, status=400)
261     if password != confirm:
262         return JsonResponse({'ok': False, 'error': 'Les mots de passe ne
correspondent pas.'}, status=400)
263     if len(password) < 8:
264         return JsonResponse({'ok': False, 'error': 'Le mot de passe doit
contenir au moins 8 caracteres.'}, status=400)
265     if User.objects.filter(username=username).exists():
266         return JsonResponse({'ok': False, 'error': 'Cet identifiant est
deja utilise.'}, status=400)
267
268     user = User.objects.create_user(username=username, email=email,
password=password)
269     login(request, user)
270     return JsonResponse({'ok': True, 'username': user.username, '
is_staff': user.is_staff})

```

A.5 Utilitaires du Scanner (scanner/utils.py)

```
1 import email
2 import re
3 import logging
4 from email.header import decode_header as _decode_header
5
6 logger = logging.getLogger(__name__)
7
8 MAX_FILE_SIZE = 10 * 1024 * 1024 # 10MB
9
10 # More robust email regex
11 EMAIL_REGEX = re.compile(r"[a-zA-Z0-9.!\#$%&'*/+=?^_`{|}~-]+@[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?(?:\.[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?)+" )
12
13 # Standard Regex to find http/https links
14 URL_PATTERN = re.compile(r'(https?://[^\s>"\']+)')
15
16
17 def _header_to_str(value) -> str:
18     """Decode an email header value (str or Header object) to a plain
19     unicode string."""
20     if not value:
21         return ""
22     try:
23         decoded_parts = _decode_header(str(value))
24         result = []
25         for raw, charset in decoded_parts:
26             if isinstance(raw, bytes):
27                 result.append(raw.decode(charset or "utf-8", errors="
28 replace"))
29             else:
30                 result.append(raw)
31         return " ".join(result)
32     except Exception as e:
33         logger.warning(f"Failed to decode header: {e}")
34         return str(value)
35
36
37 def sanitize_url(url):
38     """Sanitize URLs to prevent XSS when rendered."""
39     url = url.strip()
40     if url.lower().startswith(("javascript:", "data:", "vbscript:")):
41         return ""
42     return url
```



```

41
42
43 def get_text_from_part(p):
44     payload = p.get_payload(decode=True)
45     if isinstance(payload, bytes):
46         charset = p.get_content_charset() or "utf-8"
47         try:
48             return payload.decode(charset, errors="ignore")
49         except LookupError:
50             return payload.decode("utf-8", errors="ignore")
51     elif isinstance(payload, str):
52         return payload
53     return ""
54
55
56 def parse_eml_in_memory(file_obj):
57     """
58     Takes an in-memory Django file object, parses the .eml data,
59     and returns a dictionary of ML-ready features without saving to disk
60     .
61     """
62     file_obj.seek(0, 2)
63     file_size = file_obj.tell()
64     if file_size > MAX_FILE_SIZE:
65         raise ValueError("Fichier trop volumineux (max 10 Mo).")
66     file_obj.seek(0)
67
68     # Read enough bytes to detect at least one RFC 2822 header field
69     header_sample = file_obj.read(512)
70     file_obj.seek(0)
71     if not re.search(rb'(?i)(from|to|subject|mime-version|received|date|
72 content-type|message-id)\s*:', header_sample):
73         raise ValueError("Format email invalide.")
74
75     # 1. Read the file completely in memory
76     try:
77         msg = email.message_from_bytes(file_obj.read())
78     except Exception as e:
79         logger.error(f"Failed to parse email from bytes: {e}")
80         raise ValueError("Erreur lors de la lecture du fichier email.")
81
82     # 2. Extract Sender Domain, full email address, and Subject
83     sender = _header_to_str(msg.get("From", ""))
84     subject = _header_to_str(msg.get("Subject", ""))
85     sender_domain = ""
86     sender_email = ""

```

```

86     email_match = EMAIL_REGEX.search(sender)
87     if email_match:
88         sender_email = email_match.group(0)
89         if "@" in sender_email:
90             sender_domain = sender_email.split("@", 1)[1]
91
92     # 3. Extract the Body Text
93     body_text = ""
94     if msg.is_multipart():
95         # Loop through email parts (handles emails that have both HTML
96         and Plain Text)
97         for part in msg.walk():
98             content_type = part.get_content_type()
99             disposition = str(part.get("Content-Disposition"))
100
101             # We only want text, no attachments
102             if (
103                 content_type in ["text/plain", "text/html"]
104                 and "attachment" not in disposition
105             ):
106                 try:
107                     body_text += get_text_from_part(part) + " "
108                 except Exception as e:
109                     logger.debug(f"Failed to parse part: {e}")
110             else:
111                 try:
112                     body_text = get_text_from_part(msg)
113                 except Exception as e:
114                     logger.debug(f"Failed to parse single payload: {e}")
115
116     # Clean up the body text (remove excessive newlines/spaces)
117     body_text = re.sub(r"\s+", " ", body_text).strip()
118
119     # 4. Extract URLs from the body
120     extracted_urls = list(
121         set(URL_PATTERN.findall(body_text))
122     ) # set() removes duplicates
123
124     return {
125         "sender_domain": sender_domain,
126         "sender_email": sender_email,
127         "subject": subject,
128         "body_text": body_text,
129         "urls": [url for url in (sanitize_url(url) for url in
130 extracted_urls) if url],
131     }

```

A.6 Middleware du Scanner (scanner/middleware.py)

```
1 import time
2 import logging
3
4 logger = logging.getLogger(__name__)
5
6
7 class PipelineBenchmarkMiddleware:
8     def __init__(self, get_response):
9         self.get_response = get_response
10
11     def __call__(self, request):
12         start = time.perf_counter()
13         response = self.get_response(request)
14         elapsed = time.perf_counter() - start
15         logger.info(f"[Benchmark] {request.method} {request.path} - {
elapsed:.3f}s")
16         return response
```